Implementação de Baixo Nível da Biblioteca CKKS para Aprendizado Federado Preservador de Privacidade

Síntese técnica

Yuri Dimitre Dias de Faria
Instituto de Computação – UNICAMP
Orientador: Prof. Dr. Allan Mariano de Souza

Junho de 2026

1 Introdução

Este documento sintetiza a arquitetura de baixo nível do *backend* C++ da biblioteca CKKS customizada, desenvolvida para suportar operações de cifragem homomórfica em cenários de Aprendizado Federado (FL). O material pressupõe familiaridade com o pano de fundo do esquema CKKS, com o modelo de erro da agregação homomórfica e com o problema RLWE, cobertos no Capítulo 4 da dissertação associada [1, 5]; o foco aqui é a tradução dessas estruturas matemáticas em código executável de alto desempenho.

O núcleo da biblioteca, denominado `ckks_core`, é implementado em C++17 e organizado em seis módulos hierárquicos (tabela 1). Todas as unidades de tradução são compiladas para uma biblioteca estática com as diretivas `-O3 -march=native -ffast-math -fopenmp-simd`.

Tabela 1: Módulos do núcleo `ckks_core`.

Módulo	**Arquivos principais**	**Responsabilidade**
1 – Aritmética	`math_utils.h/.cpp`	`mod_add/sub`, Montgomery, pri
2 – Polinômio	`poly.h/.cpp`	Tipo `Poly`, adição vetorizada
3 – NTT	`ntt.h/.cpp`	Transformada negacíclica $O(N$
4 – Amostragem	`sampler.h/.cpp`	Gaussiana discreta, distribuiçõe
5 – Chaves/Cifragem	`keygen.h`, `ckks_core.cpp`	Geração de chaves, `encrypt`, `de`
6 – Avaliação	`evaluator.h/.cpp`, `encoder.h/.cpp`, `serialization.cpp`	Adição, codificação, serialização

Em contraste com implementações de propósito geral como Microsoft SEAL [5], o `ckks_core` adota duas simplificações intencionais, motivadas pelo foco exclusivo em agregação federada: (i) utiliza um **único primo NTT** $q \equiv 1 \pmod{2N}$, com $q < 2^{62}$, em lugar da cadeia RNS $Q = \prod_j q_j$ do modelo completo discutido no Capítulo 4; (ii) opera em **regime somente-aditivo**, sem *rescaling*, relinearização, chaves de Galois nem *bootstrapping*. O orçamento de ruído, portanto, não decresce ao longo das rodadas de agregação.

2 Aritmética Modular e Redução de Montgomery

Anel de polinômios. A biblioteca opera sobre o anel quociente

$$\mathcal{R}_q = \mathbb{Z}_q[X] \,/\, (X^N + 1), \tag{1}$$

onde q é um primo e N é uma potência de dois. A relação $X^N \equiv -1 \pmod{X^N+1}$ confere ao anel estrutura negacíclica: o produto de polinômios é reduzido módulo X^N+1 implicitamente pela NTT, descrita na seção 3.

Aritmética sem desvios condicionais. Adição e subtração modulares são implementadas sem desvios (*branchless*) por meio do truque da máscara binária. Para $a, b \in [0, q)$:

$$\texttt{mod_add}(a, b, q) = (a + b) - \big(q \;\&\; -(a + b \geq q)\big),$$

onde a negação do resultado booleano em 64 bits produz a máscara 0 ou $2^{64} - 1$. A subtração `mod_sub` usa raciocínio simétrico. Ambas são declaradas `inline noexcept` e mapeiam diretamente a instruções AVX2 (`vpsubq`/`vpcmpgtq`/`vpand`) sob `-march=native`.

Redução de Montgomery. Multiplicações modulares no caminho crítico (borboletas NTT, cifragem) empregam a redução de Montgomery [2] com $R = 2^{64}$. Cada elemento $a \in \mathbb{Z}_q$ é representado na *forma de Montgomery* $\hat{a} = a \cdot R \bmod q$; a multiplicação $\widehat{a \cdot b} = \texttt{REDC}(\hat{a} \cdot \hat{b})$ nunca realiza divisão por q, substituindo-a por um deslocamento de 64 bits e uma adição. As três constantes do contexto `MontgomeryCtx` ($q_{\text{inv}} = -q^{-1} \bmod 2^{64}$, $r_2 = R^2 \bmod q$ e q) são pré-computadas uma única vez por par (N, q). O valor q_{inv} é obtido por cinco iterações de Newton $x \leftarrow x(2 - qx)$ sobre $\mathbb{Z}/2^{64}\mathbb{Z}$; o valor r_2 via redução de 128 bits. A fig. 1 ilustra o padrão de uso: as conversões ocorrem uma única vez para cada conjunto de operandos, enquanto as multiplicações intermediárias permanecem inteiramente no domínio de Montgomery.

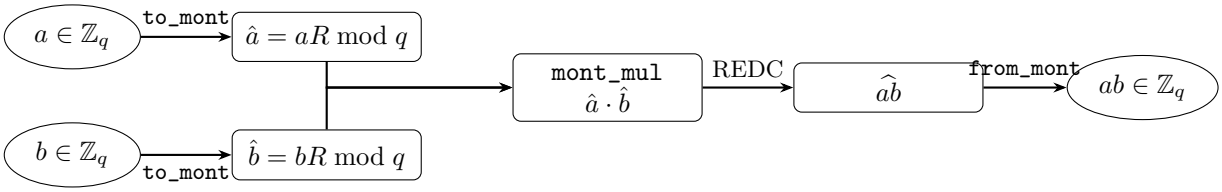


Figura 1: Fluxo de uso do domínio de Montgomery. Conversões ocorrem uma única vez; multiplicações intermediárias usam `mont_mul` sem divisão por q.

Geração de primos NTT-amigáveis. A função `generate_proth_prime(bit_size, N)` encontra o maior primo da forma $q = 2Nk + 1$ com exatamente `bit_size` bits (≤ 62), garantindo $q \equiv 1 \pmod{2N}$ e $q < 2^{62}$. A busca é descendente em passos de $2N$; cada candidato é submetido ao teste de Miller–Rabin determinístico com 12 testemunhas fixas, suficientes para cobrir todo o intervalo $[0, 2^{62})$ sem falsos positivos [6].

3 Transformada Numérica Teórica Negacíclica

A multiplicação de polinômios em $\mathcal{R}_q$ domina o custo computacional da cifragem e da decifragem. A Transformada Numérica Teórica (NTT) reduz a complexidade de $O(N^2)$ para $O(N \log N)$ [3]. A variante *negacíclica* avalia os polinômios nas N raízes primitivas $2N$-ésimas da unidade $\psi, \psi^3, \ldots, \psi^{2N-1}$, onde $\psi^N \equiv -1 \pmod{q}$, tornando a redução módulo $X^N + 1$ implícita ao algoritmo.

O contexto `NttCtx` é construído uma única vez por par (N, q) e armazena: (i) a raiz $\psi = g^{(q-1)/(2N)} \bmod q$, derivada da menor raiz primitiva g de $\mathbb{Z}_q^*$; (ii) duas tabelas de N *twiddle factors* $\texttt{psi_pow}[k] = \texttt{to_mont}(\psi^{\text{bit_rev}(k)})$ e $\texttt{psi_inv_pow}[k]$, geradas por multiplicação sequencial de Montgomery e armazenadas em **ordem bit-reversa** para acesso linear

sequencial durante as borboletas; (iii) o escalar $N_{\text{inv}}^{\text{mont}} = \texttt{to_mont}(N^{-1} \bmod q)$, usado na normalização da NTT inversa.

A NTT direta (Cooley–Tukey DIT) processa $\log_2 N$ estágios com m crescente, aplicando o fator de giro *antes* de cada soma/diferença. A NTT inversa (Gentleman–Sande DIF) processa os estágios com m decrescente, aplica o fator inverso *após* a soma/diferença e normaliza todos os coeficientes por $N_{\text{inv}}^{\text{mont}}$ ao final. Os dois algoritmos formam um par exato: nenhuma permutação adicional é necessária após a NTT inversa. Os ponteiros `__restrict__` nos laços internos informam ao compilador a ausência de aliasing, favorecendo a vetorização automática.

A multiplicação polinomial `poly_ntt_mul` aplica a NTT direta a cada operando, multiplica ponto a ponto usando aritmética de 128 bits (os resíduos de saída da NTT estão em forma normal, não de Montgomery) e aplica a NTT inversa ao resultado, com complexidade total $O(N \log N)$.

4 Geração de Chaves, Cifragem e Decifragem

Amostragem. A classe `Sampler` encapsula um gerador Mersenne Twister de 64 bits (`mt19937_64`), não copiável. Três distribuições são fornecidas: uniforme em $[0, q)$; gaussiana discreta $D_{\mathbb{Z},\sigma}$ com rejeição de amostras além de 6σ (probabilidade de rejeição $< 2 \times 10^{-9}$ por coeficiente, $\sigma = 3{,}2$); e ternária em $\{-1, 0, 1\}$, com variante de peso de Hamming fixo por Fisher–Yates parcial.

Geração de chaves. O esquema de chaves segue o paradigma RLWE [4]. A chave secreta s é um polinômio ternário com coeficientes em $\{0, 1, q{-}1\}$. A chave pública é o par $(p_0, p_1) \in \mathcal{R}_q^2$ construído como $p_1 = a$ (uniforme) e $p_0 = e - a \cdot s$, onde e é um ruído gaussiano pequeno. A invariante resultante

$$p_0 + p_1 \cdot s = e \pmod{q} \tag{2}$$

é a base da segurança RLWE: conhecendo apenas (p_0, p_1), recuperar s é computacionalmente inviável.

Cifragem. Dado um texto claro $m \in \mathcal{R}_q$ (um polinômio de coeficientes inteiros, produzido pelo codificador canônico), a cifragem sorteia um polinômio efêmero u (ternário) e dois ruídos e_0, e_1 (gaussianos) frescos a cada chamada, e calcula

$$c_1 = p_1 u + e_1, \qquad c_0 = p_0 u + e_0 + m. \tag{3}$$

As duas multiplicações polinomiais em (3) são realizadas por `poly_ntt_mul`; as somas são coeficiente a coeficiente com `mod_add`.

Decifragem. A decifragem computa simplesmente

$$m' = c_0 + c_1 \cdot s \pmod{q}. \tag{4}$$

Substituindo (3) em (4) e usando a invariante (2):

$$m' = m + \underbrace{u \cdot e_{\text{chave}} + e_0 + e_1 s}_{\varepsilon},$$

onde o termo de erro ε tem coeficientes limitados em norma infinita por $\approx 2N \cdot 6\sigma \ll q/2$, garantindo recuperação correta de m após decodificação por Δ. A fig. 2 resume o fluxo completo de cifragem, agregação e decifragem.

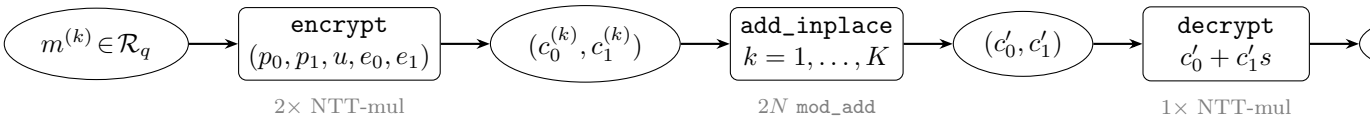


Figura 2: Pipeline de cifragem, agregação homomórfica de K clientes e decifragem. As operações dominantes são as multiplicações polinomiais via NTT ($O(N \log N)$ cada).

5 Avaliação Homomórfica e Codificação Canônica

Adição homomórfica. A classe `Evaluator` é sem estado além dos parâmetros (N, q). O método `add_inplace` realiza a adição coeficiente a coeficiente de c_0 e de c_1 de dois textos cifrados com mesmo (N, q), usando dois laços de `mod_add` anotados com `#pragma omp simd`, que sob `-fopenmp-simd` emitem instruções AVX2 vetoriais sem acionar o *runtime* de *threads* OpenMP. A correção é imediata pela linearidade:

$$(c_0^{(1)} + c_0^{(2)}) + (c_1^{(1)} + c_1^{(2)}) \cdot s = (c_0^{(1)} + c_1^{(1)} s) + (c_0^{(2)} + c_1^{(2)} s) \approx m_1 + m_2.$$

O ruído acumulado após K adições de textos independentes cresce em norma RMS como $\sqrt{K}\,\sigma_e$, produzindo erro por *slot*

$$\varepsilon_K \approx \frac{\sqrt{K}\,\sigma_e}{\Delta}, \tag{5}$$

que coincide com a expressão $\sqrt{K} \cdot e/\Delta$ da análise de agregação do Capítulo 4 da dissertação.

Codificação canônica CKKS. O codificador implementa o mapa de imersão canônica σ [1]: um vetor de $N/2$ valores complexos (os *slots*) é associado a um polinômio de $\mathcal{R}_q$ por avaliação nas N raízes primitivas $2N$-ésimas $\omega_j = e^{i\pi(2j+1)/N}$. A simetria hermitiana $\omega_{N-1-j} = \overline{\omega_j}$ garante que os coeficientes do polinômio resultante sejam reais. Na codificação (σ^{-1}), o vetor de slots é simetrizado, submetido a uma FFT de Cooley–Tukey radix-2 in-place, torcido por $e^{-i\pi k/N}$, multiplicado pela escala Δ e arredondado para inteiros em $[0, q)$. Na decodificação (σ), os coeficientes são centralizados em $(-q/2,\, q/2]$, divididos por Δ, torcidos por $e^{i\pi k/N}$ e submetidos à IFFT. O erro de quantização por *slot* é $\sim N/(2\Delta)$.

6 Serialização, Compilação e Síntese

Serialização. O módulo de serialização adota um formato binário *little-endian* auto-descritivo. A unidade atômica é a representação de um polinômio: [N: u32, 4 B] [q: u64, 8 B] [$N \times$ u64, $8N$ B], totalizando $12 + 8N$ bytes. Um `Ciphertext` ou uma `PublicKey` consistem em dois desses blocos ($2(12 + 8N)$ bytes); uma `SecretKey` consiste em um único bloco ($12 + 8N$ bytes). Cada polinômio carrega seu próprio N e q, permitindo validação completa de integridade na desserialização: tamanho do *buffer*, concordância de (N, q) e ausência de bytes residuais.

Compilação e vetorização. O núcleo `ckks_core` é compilado como biblioteca estática com código independente de posição (`-fPIC`). As diretivas de compilação são: `-O3` (otimizações agressivas), `-march=native` (instruções AVX2), `-ffast-math` (otimizações de ponto flutuante, utilizadas principalmente pelo codificador) e `-fopenmp-simd` (ativa `#pragma omp simd` sem o *runtime* de *threads* OpenMP, reservado para paralelização futura da NTT). Existe uma divergência entre os dois caminhos de construção: o `setup.py` (modo *pip*) usa apenas `-O3 -march=native -Wall`, omitindo `-ffast-math` e `-fopenmp-simd`.

Síntese e trabalhos futuros. Os três pilares técnicos do `ckks_core` são: (i) redução de Montgomery, que elimina divisões do caminho crítico; (ii) NTT negacíclica $O(N \log N)$ com tabelas de twiddle pré-computadas em ordem bit-reversa e em forma de Montgomery; e (iii) aritmética modular sem desvios condicionais, que viabiliza a vetorização AVX2. As simplificações de projeto (primo único, regime somente-aditivo) mantêm a implementação concisa sem comprometer a correção para o caso de uso alvo: agregação federada de gradientes por adição homomórfica. Os trabalhos futuros imediatos incluem a paralelização por *threads* dos laços da NTT, a substituição da multiplicação escolar $O(N^2)$ de `mul_plain` por `poly_ntt_mul`, e a extensão para cadeia RNS com *rescaling*, habilitando profundidade multiplicativa para circuitos de FL mais complexos.

Referências

[1] J. H. Cheon, A. Kim, M. Kim, e Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," em *ASIACRYPT 2017*, LNCS vol. 10624, Springer, pp. 409–437.

[2] P. L. Montgomery, "Modular multiplication without trial division," *Mathematics of Computation*, vol. 44, n. 170, pp. 519–521, 1985.

[3] P. Longa e M. Naehrig, "Speeding up the number theoretic transform for faster ideal lattice-based cryptography," em *CANS 2016*, LNCS vol. 10052, Springer, pp. 124–139.

[4] V. Lyubashevsky, C. Peikert, e O. Regev, "On ideal lattices and learning with errors over rings," em *EUROCRYPT 2010*, LNCS vol. 6110, Springer, pp. 1–23.

[5] Microsoft Research, *SEAL: Simple Encrypted Arithmetic Library – Manual*, v. 4.1, 2023. https://github.com/microsoft/SEAL

[6] C. Pomerance, J. L. Selfridge, e S. S. Wagstaff Jr., "The pseudoprimes to $25 \cdot 10^9$," *Mathematics of Computation*, vol. 35, n. 151, pp. 1003–1026, 1980.